Nadel · payment accounting for garden cooperatives

# Platform owner's manual

How the platform is built and how to run it: components, running it locally, Keycloak, creating cooperatives and users, the platform operator, every screen and button of the cooperative admin, data security, server deployment, development, common problems.

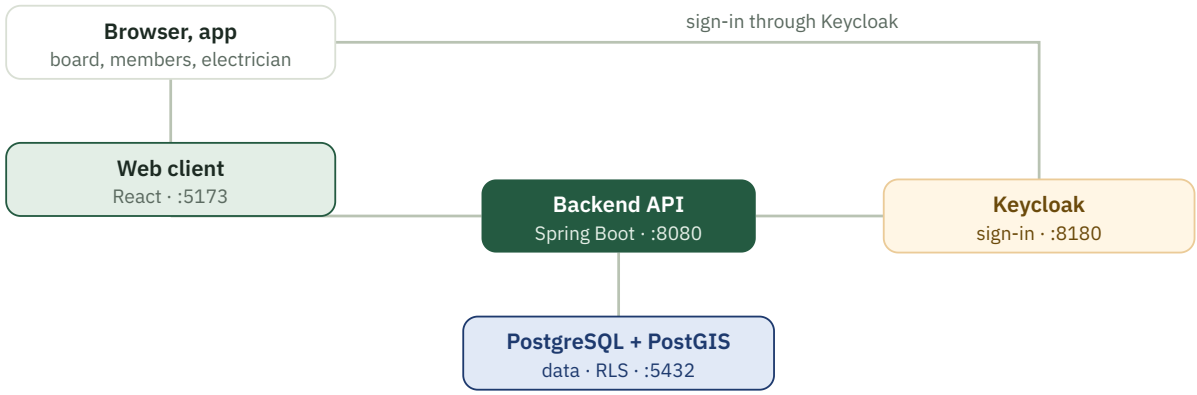
| | |
|---|---|
| Reader | The platform owner and administrator |
| State | Chapters marked "works" describe the system as it is; "planned" — decided but not built yet |
| Edition | 29 October 2026 |

# Contents


| | | |
|---|---|---|
| 1 | How the platform is built | works |
| 2 | Local environment | works |
| 3 | Keycloak: sign-in and accounts | works |
| 4 | Cooperatives: creation and isolation | works |
| 5 | The platform operator | partly |
| 6 | Cooperative admin: screens and buttons | works |
| 7 | Permissions and roles | works |
| 8 | Data, security, personal data | works |
| 9 | Server deployment | planned |
| 10 | Updates and database migrations | works |
| 11 | How the system is developed | works |
| 12 | Common problems and fixes | works |
| 13 | Decision log | |

# 1 How the platform is built

One platform serves many cooperatives. Each cooperative's data is kept apart by the database itself.




Platform components and local ports

| | |
|---|---|
| **Web client** | The board's screens and the member's account. It calculates nothing; it shows what the server returns. |
| **Backend** | One application (a modular monolith) with the modules `tenancy` (cooperatives), `identity` (sign-in and permissions), `members`, `plots`, `settings`, `imports`, `tariffs`, `meters`, `money`, `audit`. A test guards the module boundaries. |
| **API contract** | `api/openapi.yaml` is the single source of truth: the server interfaces and the web and app client are generated from it. |
| **PostgreSQL** | All data. Every table with cooperative data has a `tenant_id` and an RLS policy: a query only sees its own cooperative's rows. |
| **Keycloak** | Accounts, passwords, sign-in. Cooperative roles are not kept in Keycloak but in the database; Keycloak holds only the platform operator role. |
| **The app on a phone** | There is no separate mobile app: the web client is an installable app (a PWA). It can be added to the home screen of a phone or computer, shows notifications on the device and tells the member when there is no connection (chapter 2, "Device notifications and installation"). |

## How a request flows

1. A person signs in through Keycloak and gets a token with the `tenant_id` attribute (their cooperative).
2. The backend checks the token, finds the member by email and takes their permissions from the database.

3. For the request the database receives the `tenant_id`; RLS lets through only that cooperative's rows.
4. Board changes are published as events; the audit module records them through an "outbox", so no event is lost even if the server restarts.

# 2 Local environment

## What the computer needs

Docker Desktop, JDK 25, Node.js 22 and pnpm, Git, the GitHub CLI ( `gh` ). Repository: `~/IdeaProjects/nadel-platform` .

## First run

```
make install        # web and shared package dependencies
make up             # database, Keycloak, backend and web in one terminal (Ctrl+C stops
backend and web)
make e2e-install    # once: the browser for end-to-end tests
```

## Make commands

| Command | What it does |
|---|---|
| `make help` | Lists every command |
| `make install` | Installs client dependencies (pnpm) |
| `make infra` | Starts PostgreSQL + PostGIS and Keycloak in Docker and waits until ready |
| `make up` / `make dev` | Infrastructure + backend (:8080) + web (:5173); creates `infra/.env` with a local master key the first time |
| `make backend` , `make web` | The backend or the web alone |
| `make down` | Stops the Docker containers (data is kept) |
| `make gen` | Regenerates the API client from `api/openapi.yaml` |
| `make test` | Backend tests (with a database container) and client tests |
| `make check` | Everything CI checks: contract, formatting, lint, tests |
| `make e2e` | End-to-end browser scenarios (Playwright) on a separate database |
| `make help-shots ID=<id>` | Screenshots of one help page (for example `ID=payments-enter` ) from the running demo app, in both languages, into `web/public/help/img/` |
| `make guide-shots` | Screenshots of all help pages ( `make up` must be running) |

| Command | What it does |
|---|---|
| `make guide` | Builds the PDFs into `docs/guide/pdf`: the board guide from the help pages in `docs/help`, the owner's manual and the leaflet from `docs/guide/<lang>/*.html` |
| `make clean` | Removes build output |

## Addresses and passwords (local development only)

| What | Address | Sign-in |
|---|---|---|
| The app | `http://localhost:5173` | demo users below |
| API | `http://localhost:8080/api/v1` | a Keycloak token |
| Keycloak admin console | `http://localhost:8180/admin/` | realm master, `admin` / `admin` |
| PostgreSQL | `localhost:5432`, database `nadel` | `postgres` / `postgres` (superuser), `nadel_owner`, `nadel_app` |
| Test mail (Mailpit) | `http://localhost:8025`, SMTP `localhost:1025` | no sign-in; letters live in memory |
| Fake Telegram | `http://localhost:8091` | no sign-in; any token except one starting with `bad` |

## Mail, Telegram and background jobs in development

Delivery (feature 017) does not leave the local environment: Mailpit catches the letters, and a fake Bot API server stands in for the real Telegram. `make infra` starts both (the services `mailpit` and `fake-telegram` in `infra/docker-compose.yml`).

- **Mailpit.** SMTP is `localhost:1025` (no encryption, no password), the web page and the API are `http://localhost:8025`; `curl localhost:8025/api/v1/messages` lists the letters. Letters live in memory and disappear after a restart. This is where the test letter of "Delivery" and the invoices with their PDF attached show up.
- **Fake Telegram** (`infra/fake-telegram/server.mjs`, port 8091) keeps the updates and the messages the bot sent per token, so two cooperatives' bots do not mix. A token starting with `bad` gets 401 (this is how "The token is not valid" is tried). Controls:

```
# a member "presses Start" with a link code (chat 555)
curl -s -X POST localhost:8091/control/updates -H 'content-type: application/json' \
  -d '{"token":"123:demo","chatId":555,"text":"/start ABC123","firstName":"Olena"}'

# what the bot sent to chat 555 (method, text, document name, buttons, time)
curl -s 'localhost:8091/control/sent?token=123:demo&chatId=555'

# a member blocked the bot (sendMessage answers 403), and undo it
curl -s -X POST localhost:8091/control/block/555
curl -s -X DELETE localhost:8091/control/block/555

# forget everything
curl -s -X POST localhost:8091/control/reset
```

- **The `NADEL_TELEGRAM_API_URL` variable** is the Bot API address. In the `dev` profile it defaults to `http://localhost:8091` (the fake server), everywhere else to `https://api.telegram.org`. To check with a real bot set `NADEL_TELEGRAM_API_URL=https://api.telegram.org`, save the real token in the demo cooperative's "Delivery" and stop the fake service (`docker compose -f infra/docker-compose.yml stop fake-telegram`). The variable is to be added to `.env.example` - the owner does it before merging; without it the defaults apply.
- **Demo data.** The `dev` profile fills the demo cooperative by itself: the mailbox points at Mailpit (sender `board@demo.local`), the bot is set up against the fake Telegram (token `123456:DEMO-fake-telegram-token`, name `nadel_demo_bot`), two members are connected (`member@demo.local` and Dmytro Kravchenko), and the invoices of the closed month are already "Sent" (one attempt) or "To print". Nothing is queued, so a start never floods Mailpit. The second demo cooperative stays unconfigured.
- **JobRunr** wakes the delivery work and recovers it after a failure. Its tables are created by Flyway (migration `V30__jobrunr.sql`, because the `nadel_app` role cannot create tables), the dashboard is off and no usage data is sent. JobRunr's tables, like `event_publication`, have no `tenant_id` and no RLS: the job arguments are only a cooperative id and a channel.
- **The bot and long polling.** The system polls each cooperative's bot itself (`getUpdates`, 25 seconds), with no public address and no webhooks. *One* application instance polls: it holds a "lease" (the cooperative's row in `bot_poll_state`, 60 seconds, by database time). A second instance waits and takes the bot over about a minute after the first stopped; a clean shutdown gives the lease up at once. Each update is taken "at most once": after a crash one reply may be lost (the member just presses again), but a meter reading from the bot is never submitted twice. If the fake server is not running the application still works, and the bot retries with a growing pause.

## Demo users (password `nadel`)

| Email | Who |
|---|---|
| `chairman@demo.local` | Chairman of CT «Надія» |
| `treasurer@demo.local` | Treasurer |

| Email | Who |
|---|---|
| `electrician@demo.local` | A member with the custom role "Електрик" |
| `member@demo.local` | A member |
| `member@other.local` | A member of another cooperative (СТ «Світанок»), to check isolation |
| `operator@nadel.local` | The platform operator (no access to cooperative data) |

## The local master key (`infra/.env`)

Members' contacts are stored encrypted (chapter 8). The first time `make up`, `make backend` or `make e2e` runs, it creates the file `infra/.env` with one line, `NADEL_PD_MASTER_KEY=…`, a random local key. The file is git-ignored, has mode 600 and is read only by the backend process. An existing file is never overwritten.

- Never commit this file and never paste the key into a chat or a ticket.
- If you delete it, the contacts already in the local database become unreadable (the backend refuses to start with `PD_MASTER_KEY_MISMATCH`, chapter 12). Then reset the local database too (below) — it is only demo data.

## Device notifications and installation

A member can install the web client on a phone (a PWA) and receive short notifications on the device: a new invoice, a payment, an accepted or rejected reading, an announcement, and the board gets new readings to review. The notifications carry no amounts and no names. This works only when two things hold: the site is opened over HTTPS (browsers make an exception for `localhost`) and the server has VAPID keys.

- **What VAPID keys are.** The server's key pair: the browser sees the public key when a device subscribes, and the private key signs every notification so that the Chrome, Firefox or Safari notification service knows who sent it. The private key is a secret, like the master encryption key.
- **Where they go.** The server environment: `NADEL_WEBPUSH_PUBLIC_KEY`, `NADEL_WEBPUSH_PRIVATE_KEY` and `NADEL_WEBPUSH_SUBJECT` (the owner's contact, `mailto:…` or `https:…`). On the stand it is the file `infra/stand/.env`. Without all three values notifications are off and the application runs on (the log shows one warning and the account has no notification switch).
- **How to get them.** The stand creates the pair itself once (`infra/stand/stand-init.sh`) and keeps it afterwards. For another server run the same `openssl` commands and put the two lines into the environment file:

```
b64url() { sed 's/../\\x&/g' | { read -r esc; printf "$esc"; } | base64 | tr '+/' '-_'
| tr -d '=\n'; }
key_text=$(openssl ecparam -name prime256v1 -genkey -noout | openssl ec -text -noout
2>/dev/null)
priv_hex=$(printf '%s\n' "$key_text" | awk '/^priv:/{f=1;next} /^pub:/{f=0} f' | tr -d
' :\n' | tail -c 64)
pub_hex=$(printf '%s\n' "$key_text" | awk '/^pub:/{f=1;next} /^ASN1 OID|^NIST
CURVE/{f=0} f' | tr -d ' :\n')
echo "NADEL_WEBPUSH_PUBLIC_KEY=$(printf '%s' "$pub_hex" | b64url)"
echo "NADEL_WEBPUSH_PRIVATE_KEY=$(printf '%s' "$priv_hex" | b64url)"
```

- **Changing the keys.** A new pair invalidates every subscription: notifications turn off on every device until the members turn them on again (the switch "Notifications on this device"). So create the keys once and do not change them without a reason.
- **In development.** `application.yml` holds a test key pair for local work only; it never reaches the stand or a production server. `make infra` also starts a fake push service (`fake-push`, port 8092): it accepts notifications only from subscriptions that point at it, decrypts nothing and forwards nothing. To see what arrived: `curl localhost:8092/control/received`; to clear it: `curl -X POST localhost:8092/control/reset`. Real notifications on a real phone need a server with HTTPS and its own keys.
- **iPhone.** Notifications work only in the installed app ("Share → Add to Home Screen") on iOS 16.4 or newer.

## Resetting the local database

```
make down && docker compose -f infra/docker-compose.yml down -v && make up
```

Erases all local data; migrations and demo data load again and Keycloak imports the realm again. Needed after changing `infra/keycloak/nadel-realm.json` or when test data has spoiled the demo.

## PostGIS image on a Mac with Apple Silicon

The default image `postgis/postgis:17-3.5` exists only for Intel (linux/amd64); on an M-series Mac Docker runs it emulated, and the database, the backend tests included, is several times slower. Both the local stack and the backend tests (Testcontainers) take the image from the environment variable `NADEL_POSTGIS_IMAGE`. On a Mac put the multi-architecture build of the same PostgreSQL 17 + PostGIS 3.5 into the shell profile (`~/.zshrc`):

```
export NADEL_POSTGIS_IMAGE=imresamu/postgis:17-3.5
```

It is built by a maintainer of the official PostGIS images from the same sources. After switching, reset the local database once (above): a data volume is not moved between processor architectures. CI and servers keep the default image.

# 3 Keycloak: sign-in and accounts

Keycloak keeps accounts and passwords. The platform's realm is `nadel` with the client `nadel-web` (the app, also when installed on a phone) and the service client `nadel-backend-admin`; the `nadel-mobile` client stays in the realm file unused. Sign-in is by email, self sign-up is off, password reset by email is allowed (it works once mail is configured), the sign-in page speaks Ukrainian (default) and English.

## How an account is tied to a cooperative

- The user attribute `tenant_id` is the cooperative's identifier. It goes into the token, and the backend works only with that cooperative's data.
- The member is found by **email**: the email in Keycloak must match the email on the member card (any letter case). The backend remembers the link at the first sign-in. The first chairman of a new cooperative is linked differently, by the Keycloak user's identifier (chapter 4).
- Roles and permissions live in the cooperative's database, not in Keycloak. The exception is the `PLATFORM_OPERATOR` role.

## Giving a member access (the board invites)

Accounts are no longer created by hand in Keycloak. The board invites a member from the member card:

1. The member exists in the system and their card has an email; the cooperative's own mailbox is set up in the settings (the chairman does this).
2. Members → the member's card → **Invite to the system** (permission "Invite to the system", the chairman by default). The system creates the Keycloak account with the `tenant_id` attribute and emails a one-time link from the cooperative's mailbox.
3. The member opens the link (valid 72 hours), sets a password on the app's page and signs in. The card shows the access status: not invited, invited, active, disabled. The board can resend the link (the older one stops working) or disable access.

Nobody can disable their own access, and the access of the last chairman who has access can be neither disabled nor removed by archiving: a cooperative always keeps a chairman who can sign in. The set-password page counts only failed links: after 5 wrong or expired ones within 15 minutes an address gets "Too many attempts" for further ones until the window ends, while a working link always opens. A member gets at most 5 invitations (first and repeated) within 15 minutes, and one address sends at most 30 (`NADEL_INVITATIONS_RATE_LIMIT_MAX_REQUESTS`, `NADEL_INVITATIONS_RATE_LIMIT_MAX_SENDS_PER_ADDRESS`, `NADEL_INVITATIONS_RATE_LIMIT_WINDOW`). Changing the e-mail of a member with access turns their account off and removes its password and sessions; the new link goes to the new address. The counters live in the backend's memory and are cleared by a restart.

## The platform's own Keycloak client

To create users the backend signs in to Keycloak as a separate **confidential** client, `nadel-backend-admin` (service account only). It has exactly three `realm-management` roles: `manage-users`, `view-users` and `query-users`.

- Its secret lives only in the environment: `NADEL_KEYCLOAK_ADMIN_CLIENT_SECRET` in `/opt/nadel/.env` (mode 600) on the server. Locally the realm file holds a dev value.
- **Rotation:** `./stand-init.sh` on the server issues a new secret and writes it to `.env`; then run `docker compose up -d backend` so the backend picks it up. Every run of the script changes the secret. Never choose the value by hand and never share it.
- A stand imported before this client existed needs a fresh realm import (Reset in `infra/stand/README.md`).

## Other tasks

| Task | Where |
|---|---|
| Reset a password | Users → user → Credentials → Reset password (Temporary) |
| Block sign-in | Users → user → switch off **Enabled**. Archiving the member in the system also blocks sign-in. |
| End all sessions | Users → user → Sessions → Sign out |
| Make someone the platform operator | Users → user → Role mapping → Assign role → `PLATFORM_OPERATOR`; no `tenant_id` attribute |
| Change realm settings for good | Edit `infra/keycloak/nadel-realm.json` and reset the local database (chapter 2): the file is imported only into an empty Keycloak |

# 4 Cooperatives: creation and isolation

A cooperative (a *tenant* in the code) is a row in the `tenant` table with an identifier (UUID) and a name. When a cooperative is created the system gives it the base roles, the settings and the standard charge types.

## Creating a new cooperative: the first-chairman command

Contacts (phones, email, address) are encrypted by the application, so SQL cannot write them. The first chairman is therefore created *without* contacts and is linked to the Keycloak account by the account's identifier; the chairman enters the email and phones in the app afterwards.

On the server, in `/opt/nadel`, with Postgres and Keycloak running:

```
./first-chairman.sh --cooperative "SK Name" --email chairman@example.org \
  --last-name Ivanenko --first-name Petro
```

It creates the cooperative (base roles, settings, charge types), the chairman and their Keycloak account, and **prints the one-time set-password link once** on the screen (a new cooperative has no mailbox yet, and Keycloak needs no mail settings). Pass the link to the chairman privately; it is valid for 72 hours and cannot be shown again. Running the command again changes nothing; `--resend` issues a new link (the old one stops working). The command is a one-shot container without a web server and exits with an error code on any failure. The chairman sets a password, signs in and enters the email and phones in the app; later members are invited from the member card.

## The old manual way (SQL, only if the command is not available)

1. Connect to the database as the superuser: `docker compose -f infra/docker-compose.yml exec postgres psql -U postgres -d nadel`.
2. Create the cooperative, with the name filled in, and note the identifier from the message:

```
DO $$
DECLARE t uuid := gen_random_uuid();
BEGIN
  INSERT INTO tenant (id, name) VALUES (t, 'СТ «Назва»');
  PERFORM seed_base_roles(t);          -- roles, settings, charge types
  RAISE NOTICE 'tenant_id = %', t;
END $$;
```

1. Create the chairman's account in Keycloak (Users → Add user, Email verified on, Credentials → temporary password) with the attribute `tenant_id` set to that identifier (needs "Unmanaged attributes: Admin can edit" in Realm settings). Open the new user and copy the **ID** shown on the **Details** tab (a UUID) — it is the Keycloak *subject*.

2. Back in `psql`, add the chairman with the identifier and the subject (name and both values filled in):

```
DO $$
DECLARE t uuid := '<tenant_id>'; m uuid := gen_random_uuid();
BEGIN
  INSERT INTO member (id, tenant_id, last_name, first_name, delivery_channel,
keycloak_subject)
  VALUES (m, t, 'Last name', 'First name', 'PAPER', '<Keycloak user ID>');
  INSERT INTO member_role (tenant_id, member_id, role_id)
  SELECT t, m, id FROM role WHERE tenant_id = t AND base_code = 'CHAIRMAN';
END $$;
```

1. The chairman signs in (the subject links the account to the card) and opens "Members" → their own card → "Edit" to enter the email, phones and the preferred channel; the app encrypts them. Then the Excel import, members, tariffs and meters.

The cooperative's encryption keys are created by the server the first time the cooperative needs them (for example when the chairman saves the first contact); there is nothing to do for them here. The delivery channel stays "Paper" until a contact exists, because the database rejects a channel without its contact.

**A FALLBACK**

This SQL is a fallback; creation through it is not in the audit log. Use the command above; the operator console comes later.

## Isolation

- The application talks to the database as `nadel_app`, which cannot bypass RLS; migrations run as `nadel_owner`.
- Every new table has an isolation test: another cooperative's data is invisible even to a direct query.
- Deleting a cooperative is not provided; the data stays until an archiving decision (phase 7).

# 5 The platform operator partly

The operator is the person who runs the platform (you). The principle (ADR 0008): **the operator does not see cooperative data** — no members, no plots, no money.

| What | State |
|---|---|
| Signing in as `operator@nadel.local`: the app says "No cooperative", data is unavailable | works |
| Operator console: list of cooperatives, creating a cooperative and its chairman, blocking | phase 7 |
| Platform reports without personal data: number of cooperatives, plots, active users, storage used, subscription state | phase 7 |
| Support access: a chairman grants temporary access to their cooperative; every action goes to their audit log | phase 7 |
| Subscriptions and payment for using the platform | phase 7 |

Until then the operator's technical tasks are Keycloak (chapter 3), creating cooperatives (chapter 4), the server and backups (chapter 9).

# 6 Cooperative admin: screens and buttons

Every screen, every button — what it does and which permission it needs. A person does not see buttons they have no permission for; the server refuses such a request anyway.

## Header and menu

| Element | What it does | Permission |
|---|---|---|
| Cooperative name with the logo | Goes to the user's home screen: for a cooperative member this is "My account" | — |
| УКР / ENG | Interface language, remembered in the browser | — |
| System / Light / Dark | Theme | — |
| Sign out | Ends the session | — |
| Menu | Shows only the sections the person has permissions for. An ordinary cooperative member has no "Plots" and no "Help": their menu starts with "My account", and opening the plot list by its address sends them to the account. "My account" is in the menu only for someone who owns a plot now (an electrician without a plot sees "Readings" and "Meters"). The home screen is the first menu item. The "Help" item is there only for those who hold at least one permission | — |
| "?" in the header | Opens help about the current screen in a side panel (details below, "Help in the application"). A cooperative member opens the help with this button | — |

## Plots

| Element | What it does | Permission |
|---|---|---|
| Search, "Kind", "Status", "Show archived" | Filter the register | View plots (a cooperative member does not see the plot list) |
| Add plot | Form: number, street, kind, area (sotkas or m²), cadastral number, building, notes | Edit plots |
| Card → Edit | Changes the plot's data | Edit plots |

| Element | What it does | Permission |
|---|---|---|
| Card → Archive / Restore | Hides the plot from the list or brings it back. Refused while the plot has a lasting water connection (or an active meter): end the connection on the plot card first. The same applies to turning a plot into common land | Edit plots |
| Card → Delete | Only for a plot that never had an owner | Edit plots |
| Card → Change owner | Date, new owners with shares (sum = 1), basis; "Add member" right in the form | Edit ownership |
| Card → Correct (next to a record) | Fixes dates, share or person in an ownership record, with a reason | Edit ownership |
| Card → Tariffs (month) | What the plot will be charged for the chosen month | View tariffs, or own plot |
| Card → Meters | The plot's meters with history and replacement chain | View meters, or own plot |

## Members

| Element | What it does | Permission |
|---|---|---|
| Search, "Show archived" | By any part of the name; by the full phone in any usual form or exactly its last 4 digits; by the whole email. A part of an email or digits from the middle of a phone find nothing | View members |
| Add member | Name, phones, email, address, preferred channel, Telegram, Viber, notes | Edit members |
| Card → Edit | Changes the member's data | Edit members |
| Card → Archive / Restore | Blocks or restores sign-in; not for a plot owner or the last chairman | Edit members |
| Card → Change roles | Assigns roles to the member | Manage roles |
| Card → Change history | The audit log of this member | View audit log |

## Roles

| Element | What it does | Permission |
|---|---|---|
| Add role | Name, description, set of permissions | Manage roles |
| Edit | Changes name, description, permissions (the chairman's permissions never change) | Manage roles |
| Restore standard name / description | Undoes the renaming of a base role | Manage roles |
| Delete | Only a custom role assigned to nobody | Manage roles |

## Cooperative settings

| Element | What it does | Permission |
|---|---|---|
| Who sees plot colours | "Own plots only" or "All plots" | Manage settings |
| Photo of a reading is required for members | The panel "Meter readings". When on, a member cannot submit a reading without a photo of the meter (JPEG, PNG; an iPhone HEIC photo is converted by the phone itself); the electrician and the board add a photo if they wish | Manage settings |
| Meter tree | Opens the tree (below) | View meters |
| Tree → From / To | A range: every meter shows consumption, "calculated" or "no reading after …" | View meters |
| Tree → Move | A new parent meter (or "outside the tree") from the date in "Effective from" | Manage meters |

## Import

| Element | What it does | Permission |
|---|---|---|
| Download template | The Excel template in the interface language | Run import |
| Choose file → Check file | Checks without saving: new, updated, unchanged, problems by sheet, row and column | Run import |
| Import | Saves everything at once (only with no problems, within 30 minutes) | Run import |
| Import history | Who, when, which file, how many records | Run import |

## Tariffs

| Element | What it does | Permission |
|---|---|---|
| Charge types → Add / Change | Name, description, "Active" (an inactive type gives no amounts) | Manage tariffs |
| Charge types → Up / Down | Order; it also sets the payment allocation priority within one debt date | Manage tariffs |
| Charge types → Archive | Removes the type and its tariffs | Manage tariffs |
| New tariff | Charge type, name, first version: valid from, basis, frequency, rate, due date, note | Manage tariffs |
| Card → New version | A new rate from a date; a past date shows a recalculation warning | Manage tariffs |
| Card → Edit / Delete a version | Only while no charge uses it; the only version cannot be deleted | Manage tariffs |
| Card → Plots → Change | "All plots" with exclusions (with a reason) or "Selected plots" | Manage tariffs |

## Meters

| Element | What it does | Permission |
|---|---|---|
| Filters, "Show removed meters" | Resource, kind, street | View meters |
| Register a meter | Kind, resource, plot or name, serial number, installation date, initial reading | Manage meters |
| Card → Edit | Serial number, name, note | Manage meters |
| Card → Replace meter | Date, old final reading, new serial and initial reading; "Fill in as a wrap-around" | Manage meters |
| Card → Remove | Removal date and final reading | Manage meters |
| Card → Delete | Only a meter without readings | Manage meters |
| Card → readings → Correct / Delete reading | While the reading is not used in a charge | Enter readings |

## Readings and approvals

| Element | What it does | Permission |
|---|---|---|
| Round date | The date for all readings of the round; defaults to today, with a hint | Enter readings |
| Resource, Street, "No reading since" | Round filters; the resource is remembered | Enter readings |
| Meter row → Save | Saves the reading (the row's date can be changed); shows consumption and warnings at once | Enter readings |
| To approve → Approve | A member's reading becomes counted | Approve readings |
| To approve → Reject | With a reason the owner sees | Approve readings |
| View charges | yes | yes — — |
| Run charges | yes | — — — |
| Reverse charges | yes | — — — |
| Close periods | yes | — — — |
| Recalculate charges | yes | — — — |

## Charges

| Element | What it does | Permission |
|---|---|---|
| Charges → Months → month | Totals, the check list, history of the month | View charges |
| Month → Charge the month | Preview, then "Book" | Run charges |
| Month → Close the month | Confirms with the check list; only the earliest open month | Close periods |
| Not charged → Retry | Retries one item | Run charges |
| Charges → Recalculations | The log of booked recalculations; "Open" shows the lines, the amounts and the reversals and adjustments created | View charges |
| Recalculations → New recalculation → From month, To month, types, plots, Preview | Shows what would change; nothing is written | Recalculate charges |

| Element | What it does | Permission |
|---|---|---|
| Preview → Reason → Book the recalculation | Books exactly this preview; the reason is required | Recalculate charges |
| Tariff card → Edit / Delete a used version | A preview step with a reason; the change and the recalculation are saved together | Manage tariffs and Recalculate charges |
| Meter card → Correct / Delete a charged reading | A preview step with a reason; without the permission: "The reading has already been charged — ask the chairman" | Enter readings and Recalculate charges |
| Plot account → Adjustments only | Filter of the adjustments of closed months, with links to the original charge and the recalculation | View charges (or own plot) |
| Losses / Common consumers → From, To, Preview, Book | A range run: preview, then booking of exactly that preview | Run charges |
| Past runs → Details, Reverse the run | The lines of a run; reversal with a reason | View charges / Reverse charges |
| Plot card → Account → Details | The calculation of a charge | View charges (or own plot) |
| Account → Reverse | A reversal with a reason (not offered in a closed month) | Reverse charges |
| Account → Enter an opening balance | Charge type, amount, date, reason | Run charges |
| Plot card → Water → Connect / Disconnect | A water connection with its dates | Manage meters |
| Settings → red threshold, Losses, Common consumers, Water supply | Thresholds, losses mode, splits, bulk water connections | Manage settings / Manage meters |

**KNOWN GAPS**

- The lasting connections of a removed well can only be ended on each plot's card.

## Payments and cash desk

| Element | What it does | Permission |
|---|---|---|
| Payments (menu) → list | Payments, newest first: receipt or bank document, date, plot, payer, source, amount, "Allocated", "Prepayment", a "Reversed" mark. Filters: "From", "To", "Source", "Plot", "Billing month" | View payments |

| Element | What it does | Permission |
|---|---|---|
| Payments → Enter payment | Opens the payment form | Enter payments |
| Plot card → Enter payment | The same form with the plot already chosen | Enter payments |
| Form → Plot, Source (Cash / Bank), Date received, Amount | The date is not later than today; a date other than today only shows a gentle warning; the amount is above zero with at most two decimals. Common land and an archived plot take no payments | Enter payments |
| Form → Payer, Payer note, Bank document number | The payer is optional (the plot's owners first); the note, up to 200 characters, is stored encrypted; the document number, up to 64 characters, is for bank payments only | Enter payments |
| Form → allocation table (Allocate, Add a charge, Prepayment) | The proposal: the oldest debt first, and within one debt date the order of charge types. A line's amount can be changed or another open charge added; with automatic allocation on, the free remainder goes at once to the oldest debts (a note under the "Prepayment" line says "The remainder will be placed automatically on the oldest debts"), otherwise it is the "Prepayment". The table stays on screen: a message at a wrong line and "Allocated X of Y" below | Enter payments |
| Form → Several plots of one payer | Split mode: the payer, their current plots with debts and the parts of the payment (by default in proportion to the debts); the parts must add up to the payment amount | Enter payments |
| Form → Save | For cash shows "Receipt № ..." large; "Another payment", "Open the payment card". If the data changed after the preview, "The data changed after the preview" and nothing is saved | Enter payments |
| Payment card | The payment's details, "Currently allocated" (the current allocation: charge, period, amount; "Allocated from prepayment" for automatic ones), "Allocation history" (allocations and "Taken back from the charge" with the cause and the author), the other parts of the group, who entered it and when; "Allocate prepayment" when the prepayment is above 0.00, a charge is open and automatic allocation is off; the payer note is shown only to someone who also has "View members" | View payments |
| Payment card → Reallocate | Shows the current allocation and a new proposal; saving takes back the old allocation and writes the new one in one step; written to the audit log. Lines left at 0.00 create no prepayment while open debts exist (automatic allocation on) | Enter payments |

| Element | What it does | Permission |
|---|---|---|
| Payment card → Reverse | Needs a reason (up to 500 characters); takes back every allocation of the payment, so the debts return. A payment is reversed once; a part of a group on its own; the receipt number stays and the payment is marked "Reversed" | Reverse payments |
| Plot account → payments, "Payments only" | Payments, their reversals and "Allocated from prepayment" interleaved with the charges by date; per charge: charged, paid, open; the debt after payments, with "Prepayment X" next to the total debt when the prepayment is above 0.00; "Payments only" keeps only the payments | View charges (payments: also View payments or an own plot) |
| Plot account → Prepayment → Allocate prepayment | Shown (also on the payment card) when the prepayment is above 0.00, a charge is open and "Allocate prepayment automatically" is off: a proposal over the open charges, which can be changed | Enter payments |
| Settings → Allocate prepayment automatically → Save the payment settings | On by default: prepayment never waits while the plot has open debts - the remainder of a payment is placed at once - and later goes by itself to the plot's new charges and to released amounts. Without the permission the switch is read-only. Switching it on does not allocate old prepayment at once | Manage settings |
| Charges → Months → "Received" | Next to "Charged": what came in during the month (payments net of reversals) | View charges |
| Reports → "Payments" | Payments by billing months or receipt dates; filters: source, street; totals per source; the payer note only for someone who also has "View members". The "Plot balances", "Debtors" and "Charges by month" reports now take payments into account | View reports + View payments |
| My account → My charges | A member sees payments of their own plots only: date, receipt number, amount, what it paid, the prepayment and the debt after payments; never the note or who entered the payment | own plot |

## Cash and bank

| Element | What it does | Permission |
|---|---|---|
| Cash and bank (menu) | Two cards (cash, bank account) with today's balances, warnings about a negative balance, the last count and the "Unresolved difference", the reconciliation months with their status; the account journal for a period | View cash and bank |
| Account card → "Enter the opening balance" | An amount and a date, once; it cannot be changed (a mistake is corrected by a cash count or a bank difference) | Approve cash book operations |

| Element | What it does | Permission |
|---|---|---|
| "Expense", "Hand over to the bank", "Other income", "Cash count" | Entry forms. An expense takes up to 5 files (JPEG/PNG photos or PDF, up to 5 MB), with a "Take a photo" button on a phone. The cash cannot become negative on the date of the entry or later | Enter cash book entries |
| Journal → "Reverse"; expense → "Reverse and record again"; count → "Book the difference" | A reason is required; a reversal is a line of the opposite amount, the entry itself stays | Approve cash book operations |
| Expenses ("All expenses" button), an expense card | A list with the filters "From", "To", "Account", "Category"; a card with files to download | View cash and bank |
| Cash count → "Cash count act" | A PDF in Ukrainian: book balance, counted balance, difference, signatures of the treasurer and the chairman | View cash and bank |
| A month on the bank card → the "Bank reconciliation" screen | The four statement figures, the comparison with the books, the bank lines of the month; "Save the statement", "Reconciled" (only when the closing balance difference is 0.00), "Bank difference" | Enter cash book entries ("Bank difference" - Approve cash book operations) |
| Settings → "Expense and income categories" | Add, rename, reorder, archive and restore categories (two separate lists) | Approve cash book operations |
| Plot card → "Reconciliation act" tab; My account → "Reconciliation act" | A PDF reconciliation act for a period; a member only for their own plots | View charges (a member - their own plot) |

## Invoices, certificates and receipts

| Element | What it does | Permission |
|---|---|---|
| Charges → Months → month → "Invoices of the month" → preview and "Build invoices for ..." | Preview (plots with an owner, already invoiced, and what the build will do: to be built, new versions, unchanged, skipped - as the result will say), the switch "Include invoices with nothing due", then a build while you wait (a spinner, then the result in place of the preview: built, new versions, unchanged, skipped). Refused with a message when the IBAN or the EDRPOU code is missing, or when a build of the month is already running. Recorded in the audit log | Issue invoices |
| Month → list of invoices, "PDF for printing" | One row per plot (the latest version; older ones under "Earlier versions (N)"): plot, version, amount due, issued, a PDF per row. "PDF for printing": the latest invoices on A4 pages ordered by street and plot, for paper delivery members or (a switch) all | View charges; printing: Issue invoices |

| Element | What it does | Permission |
|---|---|---|
| Plot card → "Invoices" → "Invoice for today" | The plot's invoices: the latest version of each month, older ones under "Earlier versions (N)"; builds an invoice as of today for the current billing month and downloads its PDF; a new version only when something changed, otherwise the latest version's PDF | View charges; build: Issue invoices |
| Plot card → "Certificate" → "Issue certificate" | A certificate of settlements as of the chosen date, numbered Д-year-N; the PDF opens after issue. Recorded in the audit log | Issue certificates |
| Menu "Certificate register" | All certificates with a status filter; the PDF; "Annul" with a required reason (an annulled certificate keeps its number, its PDF is marked "ANNULLED"; annulling twice is refused). Recorded in the audit log | View charges; annul: Issue certificates |
| Payment card / the screen after saving cash → "Print receipt" | A cash receipt PDF; none for a bank payment; the payer note only for those who also have "View members" | View payments |
| My account → "Invoices", "Receipt" | The latest invoice of each own plot, older months below, a PDF; the receipt of own cash payments. No certificates | own plot |

## Delivery and announcements

| Element | What it does | Permission |
|---|---|---|
| Settings → "Delivery" → e-mail: server, port, encryption, user, password, sender; "Save the mailbox" | Saves the cooperative's mailbox. The password is sealed with the cooperative's key and never shown again ("saved"); an empty field keeps the old password. Written to the change history (the field "changed", without the value) | Manage cooperative settings |
| "Send a test letter" | A letter to the user's own address from the member card; shows "accepted by the server" or the refusal in plain words and the server's reply. Works with the saved settings | Manage cooperative settings |
| "Telegram bot" → "Bot token", "Save the token", "Check the bot" | The token is sealed; only its last 4 characters are shown. The check asks Telegram and shows the bot's `@name` or "The token is not valid" / "Telegram is unavailable". A new token restarts the cooperative's bot within about a minute | Manage cooperative settings |
| "General options": "Letters per second" (1-10, default 1), "Notify about payments", "Save the options" | The letter rate and the messages about payments and their reversals | Manage cooperative settings |

| Element | What it does | Permission |
|---|---|---|
| My account → "Notifications" | The preferred way among the available ones ("Save"), "Connect Telegram" (a one-time link for 30 minutes), "Disconnect Telegram". Without a bot there is no connect button | own data |
| Member card → "Telegram" | Only "connected" or "not connected" | View members |
| Charges → Months → a month → "Send invoices for …" → "Also send invoices with nothing due", "Send" | A preview (Telegram, e-mail, paper), then queueing. Next to an invoice in the list is "Send" for the owners of one plot. "PDF for print" now prints one copy per paper recipient with the QR code "Отримувати рахунки в Telegram" (a code valid 90 days). Written to the change history ("Invoices sent") | Issue invoices |
| "Deliveries" menu → filters "Month", "Kind", "Status", "Channel", "Send again", "Printed" | The summary by channel and the list with reasons. "Send again" repeats a send from "Sent" or "Failed"; "Printed" removes the "to print" mark from the month's paper invoices. These actions are not written to the change history | Issue invoices |
| "Announcements" menu → "New announcement" → "Preview", "Send" | The list of past announcements and a card with the text and its deliveries. A new one: text up to 2000 characters, "All members" or streets, channels Telegram / e-mail, a preview with the list of members "on paper". Written to the change history ("Announcement sent") | Send announcements |

## Documents: requisites, fonts, the PDF engine

- **Requisites.** The IBAN, the name, the address and the bank are entered by the chairman in Settings. The **EDRPOU code** belongs to the cooperative's own record and only the platform operator sets it; until the operator console exists, the owner writes it into the `edrpou` column of the cooperative's `tenant` row as the database owner. Invoices are not built without both.
- **Language.** Documents are always in Ukrainian, whatever the language of the user's screens: they are official documents. Only the screens follow the language the user chose.
- **Fonts.** PDFs embed a subset of **IBM Plex Sans** (the font of the app's design), licensed under the SIL Open Font License 1.1. The font files and the licence text are in the backend resources and must travel together. Cyrillic and the hryvnia sign are readable without any font on the reader's computer.
- **The PDF engine** is Apache PDFBox. A PDF is not stored: it is drawn on each download from the data kept at build time, and the same version always gives the same file (apart from the technical creation date). The QR image is made with ZXing; the layout follows the National Bank of Ukraine rules (format 003).
- **The build limit.** A month's invoices are built in one request while the treasurer waits. The measured limit for the largest supported cooperative: see ADR 0017 ( `docs/adr/0017-documents.md` ).

- **Permissions.** "Issue invoices" (chairman and treasurer by default) builds, re-issues and prints invoices; "Issue certificates" (chairman) issues and annuls certificates. Viewing needs "View charges", and printing a receipt "View payments". Custom roles can be given both on the roles screen.

## Reports

| Element | What it does | Permission |
|---|---|---|
| Reports → "Cash book", "Bank movements", "Income and expense", "Expenses" | The four cash book reports; "View cash and bank" is enough, "View reports" is not needed | View cash and bank |
| Reports (menu) → a report card | Opens one of the nine reports; "Debtors" is there only with "View members" | View reports |
| Report → filters, "Show" | Refreshes the preview (first 200 rows and totals); under the filters it says what the period counts by | View reports |
| Report → "Download Excel" | A file with all rows in the interface language; more than 20 000 rows on a sheet is refused; recorded in the audit log | View reports |
| Reports → "Report templates" | The templates page (below) | Manage report templates |
| Reports → "Generate" next to a template | The "Generate a file" window: "From", "To" (up to a year), "Generate and download"; the file is not stored | View reports |
| Templates → "Tax report" → "Upload the template" | A fixed slot; until it is uploaded the status is "The template is not added yet" | Manage report templates |
| Templates → "Add a template" | A name, a description, an .xlsx file up to 2 MB; refuses macros, damaged and oversize files | Manage report templates |
| Template → "Upload new version", "Versions" → "Download" | The new version becomes current; older versions stay available | Manage report templates |
| Template → "Archive" | The template becomes read-only, its name is freed, its versions can be downloaded | Manage report templates |
| Templates → "Placeholders" (search, "Copy") | The catalogue of every placeholder: type, description, example | Manage report templates |

| Element | What it does | Permission |
|---|---|---|
| Settings → "Requisites" → "Save the requisites" | Official name, legal address, bank, IBAN (validated); the EDRPOU code is read-only (it is set by the platform operator, see below); recorded in the audit log. While the IBAN or the EDRPOU code is missing a warning banner is shown here and on the invoice screens | Manage settings |

**DEMO DATA**

The sample template "Відомість земельного податку" (Land tax statement) and the made-up requisites (EDRPOU, address, IBAN) exist only in the demo data of the local profile ( `demoledger=on` ), so that "Generate" works at once. The "Tax report" slot is left empty in the demo on purpose. None of this is present in tests or on a server.

## Audit log and account

| Element | What it does | Permission |
|---|---|---|
| Audit log → record type, From, To, "Only this record", "Only this author" | Filter entries; "Remove filter". Old and new values of phones, email, address, Telegram, Viber and notes are shown only to a person who also has "View members"; for anyone else the entry says the field changed and shows "hidden" in place of the values | View audit log (values: also View members) |
| My account | My details, roles, what I may do, my plots with shares, what I will be charged, my payments and prepayment | — |
| My meters → Submit a reading | A reading for the board to approve | own plot |

## Help in the application

The help is built into the application: it has no server or database of its own. It is the pages `docs/help/uk/*.md` and `docs/help/en/*.md`, compiled together with the web client (the format is described in `docs/help/README.md`).

| Element | What it does | Permission |
|---|---|---|
| "?" in the header | Opens a panel at the side with help about the current screen. The panel is not modal: the screen stays usable and scrollable. Moving to another screen shows that screen's help in the panel, or closes it if that screen has no help. Esc and "Close" close the panel; "Open in full" leads to the help page. On a phone the panel is full screen with a "← Back" button. If the screen has no page, the button is not shown | — |

| Element | What it does | Permission |
|---|---|---|
| Help → contents and search | The `/help` section: the contents by topic and a search over the pages (it tolerates one typo in a word). A person sees only the pages for their permissions and roles; a direct link to another page shows a message that the section is not available, without the content | anyone who holds at least one permission (the menu item); everyone else uses "?" |
| Help without signing in | The pages `/help` and `/help/<id>` open without signing in, with no request to the API and no personal data; after signing in from the help the person returns to the same page | — |

Search engines: the application itself carries `<meta name="robots" content="noindex">` and is not indexed, while the (Ukrainian) help pages get their own title, description, canonical address and `robots: index, follow` when the web client is built, so they can be indexed. For a stand there is the `NADEL_NOINDEX` flag (chapter 9).

# 7 Permissions and roles

| Permission | Chairman | Treasurer | Member | Electrician (demo) |
|---|---|---|---|---|
| View members | yes | yes | — | — |
| Edit members | yes | — | — | — |
| Invite to the system | yes | — | — | — |
| Manage roles | yes | — | — | — |
| View audit log | yes | yes | — | — |
| View plots | yes | yes | — | — |
| Edit plots | yes | — | — | — |
| Edit ownership | yes | — | — | — |
| Run import | yes | — | — | — |
| Manage settings | yes | — | — | — |
| View tariffs | yes | yes | — | — |
| Manage tariffs | yes | — | — | — |
| View meters | yes | yes | — | yes |
| Manage meters | yes | — | — | — |
| Enter readings | yes | — | — | yes |
| Approve readings | yes | — | — | — |
| View reports | yes | yes | — | — |
| Manage report templates | yes | — | — | — |
| View payments | yes | yes | — | — |
| Enter payments | yes | yes | — | — |
| Reverse payments | yes | — | — | — |
| Issue invoices | yes | yes | — | — |
| Issue certificates | yes | — | — | — |
| Send announcements | yes | - | - | - |

| Permission | Chairman | Treasurer | Member | Electrician (demo) |
|---|---|---|---|---|
| View cash and bank | yes | yes | — | — |
| Enter cash book entries | yes | yes | — | — |
| Approve cash book operations | yes | — | — | — |

- The chairman always holds every permission, including any new one.
- The chairman changes the treasurer's and other roles' permissions in "Roles".
- A member without permissions sees only their own data — enforced by the server.
- Charges: "View charges" opens the Charges screens and every plot account (the treasurer has it, read-only); "Run charges" runs months, losses and common consumers, retries "Not charged" and enters opening balances; "Reverse charges" reverses a charge or a run; "Close periods" closes a month; "Recalculate charges" (the chairman only) recalculates past charges and books corrections of used tariff versions and readings. A member without "View charges" sees only their own accounts.
- Reports: "View reports" (chairman, treasurer) opens the nine reports, downloads, and generates files from templates; the "Debtors" report also needs "View members". "Manage report templates" (the chairman only) uploads, replaces, downloads and archives templates, including the tax report slot. A member and the electrician have no reports by default; the chairman gives these permissions to a custom role in "Roles".
- Payments: "View payments" (chairman, treasurer) opens "Payments", the cards, the allocation and the "Payments" report; "Enter payments" (chairman, treasurer) enters payments, splits them over plots, reallocates and allocates prepayment; "Reverse payments" (the chairman only) reverses a payment - a treasurer who made a mistake asks the chairman. The "Allocate prepayment automatically" switch needs "Manage settings". A member sees the payments of their own plots in "My charges"; the electrician sees none.
- Documents: "Issue invoices" (chairman, treasurer) builds, re-issues and prints invoices; "Issue certificates" (the chairman only) issues and annuls certificates; invoices and the register of certificates are seen with "View charges", a cash receipt printed with "View payments". A member gets the invoices and receipts of their own plots only and never certificates.
- Delivery: "Manage cooperative settings" (chairman) sets up the mailbox and the bot and opens the "Delivery" panel; "Issue invoices" (chairman, treasurer) also sends invoices, opens the "Deliveries" screen, repeats sends and marks paper as printed; "Send announcements" (the chairman only) opens "Announcements" and sends them. A member chooses the way to receive invoices and connects Telegram in "Notifications" themselves, with no separate permission.
- Cash and bank: "View cash and bank" (chairman, treasurer) opens "Cash and bank", the journals, expenses with files, counts, reconciliations and the four cash book reports (these need no "View reports", so an audit committee as a custom role can have only this permission); "Enter cash book entries" (treasurer, chairman) records expenses, handovers to the bank, other income, cash counts, statement figures and "Reconciled"; "Approve cash book operations" (the chairman only) enters opening balances, books differences, reverses cash book entries and manages expense and income

categories. The reconciliation act is built with "View charges", and by a member for their own plots. A member does not see "Cash and bank".

- Access to the system: "Invite to the system" (the chairman only; thirty-two permissions in all) invites a member from the member card, resends the link, disables and enables access and shows the access status. It is used together with "View members".

# 8 Data, security, personal data

| | |
|---|---|
| **Isolation** | Database RLS on every table with cooperative data; the application cannot bypass it. |
| **Access** | The server checks permissions on every request; a member sees only their own plots. |
| **Audit log** | Who, when, what changed, old and new value. Entries are never changed or deleted. |
| **Not stored** | Tax numbers (РНОКПП), passport data, dates of birth — there are no such fields. |
| **Server logs** | No personal data: even a database error message does not include values. |
| **Events** | Completed events are deleted from the event table at once, because they may carry personal data. |
| **Secrets** | Only from environment variables; the repository holds only local development passwords. CI scans for leaked secrets (gitleaks). |
| **Expense attachments** | Photos and PDFs of receipts are stored **in the database** (an attachment table with `tenant_id`, RLS and isolation between cooperatives; files are only added, never changed), not on disk. At most 5 files of 5 MB per expense; the type is checked by content (JPEG, PNG, PDF), not by name; location (GPS) and camera data are removed from photos. A file can be downloaded only by someone who has "View cash and bank" in their own cooperative. HEIC files from an iPhone are converted to JPEG only by the phone or browser (the server does not convert HEIC); if the browser cannot open HEIC the user sees "This browser cannot open HEIC photos from an iPhone. Add the photo in Safari, or on the iPhone turn on Settings → Camera → Formats → "Most Compatible" (photos will be JPEG)." Thumbnails on the form and the expense card are made in the browser from the stored file (no extra images on the server). The counterparty and the purpose of an expense are personal data: stored protected, visible in the audit log only with "View members". Attachments make the database and its backups grow: plan the space (see chapter 10). |
| **Report templates are data** | The system **never executes** formulas or code from a template: placeholders are substituted, formulas are left alone, and Excel recalculates them in the opened file. Macros (`.xlsm`), active content, OLE objects and external links never reach storage (links are removed with a warning). Only `.xlsx` up to 2 MB is accepted; the zip-bomb guard (limits on the unpacked size and the compression ratio) and the XXE guard (XML without DTDs or external entities) apply both at upload and at every generation. Templates and versions are tables with `tenant_id`, RLS and isolation between cooperatives; generated files are not stored. |
| **Reports and personal data** | Personal data (names, phones, e-mails) is only in the "Debtors" report, which also needs "View members"; templates have none. The platform operator sees |

| | |
|---|---|
| | neither reports nor cooperative data. The audit log has the actions `REPORT_GENERATED` (the report or the template with its version, the filters, the row count — never the content), `REPORT_TEMPLATE_UPLOADED` and `REPORT_TEMPLATE_ARCHIVED`; a change of requisites is recorded with the changed fields. |
| **Cooperative secrets: the one exception** | The mailbox password and the bot token belong to the cooperative and are entered by its board, so they are cooperative data, not environment variables (ADR 0018). They are sealed with the cooperative's own key (AES-256-GCM, the master key comes from the environment), cannot be read back through the API ("saved" and the last 4 characters of the token) and never reach the change history, events or logs. The platform's own secrets (the database, the master key, Keycloak) stay in environment variables only. The platform operator does not see the cooperatives' mailboxes and bots. Losing the master key means the chairman enters the password and the token again. |
| **The cooperative's mail server: public addresses only** | The chairman types the mail server and its port, and the platform's server opens the connection, so without a limit the test letter could probe the server's own network. Outside the `dev` profile a mailbox may use only the ports 25, 465 and 587 and a server whose every address is public: not loopback, not link-local (with the cloud metadata address `169.254.169.254`), not private (RFC 1918, `fc00::/7`), not shared `100.64.0.0/10` or otherwise reserved. Otherwise saving and the test letter get 400 `SMTP_HOST_NOT_ALLOWED`, and a delivery fails without retries. The address is checked before every connection, and the connection goes to exactly the checked address (no second DNS lookup). The `dev` profile turns on `nadel.delivery.smtp-private-hosts` so that Mailpit on `localhost:1025` works. |
| **Two functions across cooperatives** | The bot poller and the minute-by-minute check of the queue must know which cooperatives have a bot or waiting messages before they have a current cooperative. Two database functions exist for that (`notifications_bot_tenants()` and `notifications_due_lanes()`): they return only cooperative ids and a channel code, never a row, an address or a text, and only `nadel_app` may run them. They are the only functions of the platform that cross cooperatives; every read of data after them runs inside its own cooperative under RLS. A new such function needs its own ADR. |
| **Where letters are sent** | The chairman types the mail server address, and the test letter connects to it from the server. This is a deliberately accepted limit (ADR 0018): only the chairman holds "Manage cooperative settings", the connection has 10-second timeouts, and the answer carries only a code and the last line of the server's reply. |
| **Dependencies** | CI checks known vulnerabilities (pnpm audit, OSV) and the code (Semgrep). |
| **Encrypted contacts** | Phones, email, postal address, Telegram, Viber and the notes of a member are stored encrypted (AES-256-GCM), also in the audit log and in the pending Excel import. Names stay readable, because the board searches and sorts by |

them. A copy of the database without the master key reveals no contact. On the server the disk and the backups are encrypted as well (chapter 9).

## Personal data encryption: keys

- **Master key** `NADEL_PD_MASTER_KEY`: 32 random bytes in base64. It lives only in the server's environment — never in the database, the repository, the logs or a backup.
- **Cooperative keys**: every cooperative has its own data key (and a key for the search indexes), kept in the database *wrapped* by the master key. They are created the first time a cooperative needs them. So a stolen database holds only wrapped keys, and one cooperative's key never opens another's data.
- Search finds contacts without decrypting the table: the server compares keyed hashes of the full phone, its last 4 digits and the whole email (chapter 6, "Members").

## Generating the master key

```
openssl rand -base64 32
```

Put the output into the server's secret store as `NADEL_PD_MASTER_KEY`. Use different keys for staging and production. The server in the `prod` profile **refuses to start** without a key or with one that is not 32 bytes of base64; the message names the setting `NADEL_PD_MASTER_KEY` and never shows a key. With a key that does not match the database it refuses too, with `PD_MASTER_KEY_MISMATCH` (chapter 12).

**A LOST MASTER KEY = LOST CONTACTS**

Nothing else can open the encrypted contacts: not a backup, not the developers. Keep at least two copies **offline** (a password manager entry and a sealed paper copy), apart from the database backups. A backup stored together with its key gives no protection, and a backup without the key cannot be restored to readable contacts. Names, plots, charges and payments are not affected — only the contacts, addresses and notes.

## Rotating the master key

1. Generate a new key: `openssl rand -base64 32`.
2. Deploy with the new key as `NADEL_PD_MASTER_KEY` and the old one as `NADEL_PD_MASTER_KEY_PREVIOUS`.
3. At start-up the server re-wraps every cooperative's keys with the new key (one transaction per cooperative) and logs counts only, for example `Personal data keys: 12 checked, 12 re-wrapped, 0 left on the previous master key`. If a key is wrapped with neither master key, the start-up stops with `PD_MASTER_KEY_MISMATCH` before anything is changed.
4. Remove `NADEL_PD_MASTER_KEY_PREVIOUS` and deploy again.

5. Keep the old key offline until no backup made before the rotation is retained: such a backup still holds keys wrapped by the old key.

Only the master key is rotated; the cooperatives' own data keys stay as they are. Restore drills need the key: restore the database, start the backend with the key and check that contacts open.

# 9 Server deployment planned · 6a

The decision (ADR 0010): the whole product is finished locally first, the server comes last. The plan is as follows.

## What runs on the server

- One virtual server in an EU data centre (initially Hetzner Cloud, Germany or Finland), about 4 vCPU and 8 GB RAM; a smaller second one for staging.
- Docker Compose: PostgreSQL + PostGIS, Keycloak in production mode, the backend and Caddy (the web client, HTTPS with automatic certificates). Only ports 80 and 443 are open to the outside.
- Cost: tens of euros per month.

## How updates arrive

1. Every merge to `main`: GitHub builds the images, pushes them to the GitHub Container Registry and updates staging.
2. A version tag `v…`: deployment to production after your approval in GitHub, with a database backup right before.
3. Rollback: deploy the previous tag with a separate action.

## First installation

- The server: SSH keys only, no root, a firewall, automatic security updates.
- A domain and DNS pointing to the server; secrets in the server's environment file and in GitHub environments; the backend refuses to start with a default password.
- The `first-chairman.sh` command creates the cooperative, its chairman and the Keycloak account and prints a one-time set-password link for you to pass to the chairman (chapter 4). The backend also needs `NADEL_KEYCLOAK_ADMIN_CLIENT_SECRET` (chapter 3).
- The Keycloak admin console and the database only through an SSH tunnel.

## HTTPS and notification keys

- Installing on a phone and notifications need HTTPS; Caddy issues the certificate itself. Without HTTPS the browser neither offers to install the app nor allows subscribing to notifications.
- The VAPID keys and the owner's contact (`NADEL_WEBPUSH_PUBLIC_KEY`, `NADEL_WEBPUSH_PRIVATE_KEY`, `NADEL_WEBPUSH_SUBJECT`) live in the server's environment file, see chapter 2. Changing the keys turns notifications off on every device until the members turn them on again.

## Help and search engines

The `NADEL_NOINDEX` setting in `.env` (default `true`) makes Caddy add the header `X-Robots-Tag: noindex, nofollow` to every response, so a stand is never indexed. Set `NADEL_NOINDEX=false` only on a real public site where the help pages (`/help/…`) should be indexed, then run `docker compose up -d caddy`. The site address for canonical links is `VITE_SITE_URL` at web build time. More: `infra/stand/README.md`.

## Backups and monitoring

- Every night an encrypted copy of the application and Keycloak databases, stored outside the server; 30 daily and 12 monthly copies kept. Every month a copy is restored on staging as a test.
- An external uptime check with Telegram alerts; error tracking without personal data; disk, memory and certificate alerts.

A step-by-step runbook with exact commands joins this chapter with the deployment feature.

# 10 Updates and database migrations

- The database structure changes only through Flyway migrations (`backend/src/main/resources/db/migration`), run when the backend starts. Existing migrations are never edited — only new ones are added.
- "Expand, then contract": a release may add tables and fields; removing or renaming waits for the next release, when the old code no longer runs. The previous version thus works on the new database and a rollback is safe.
- Demo data (`db/demo`) loads only in the development profile; the server never has it.
- Cash and bank (feature 018): the migration `V32__treasury.sql` creates the cash and bank tables (accounts, journal, expenses with attachments, counts, reconciliations, categories) with RLS, three permissions, and **carries over the existing payments**: every payment already entered and every reversal of a payment gets its journal line exactly once (a second run duplicates nothing). This is the one place where a migration reads the payments table, because it runs as `nadel_owner`; no old instance runs during the deploy, so no new payments appear at that moment (ADR 0010: deploys with downtime). `V33` adds the reason for booking a cash count difference, `V34` the table of bank differences of a month. Attachment files make the database grow, so watch its size and the backups (chapter 9).
- Uploads: one request with an expense takes up to 27 MB (5 files of 5 MB plus the JSON part), an Excel import up to 10 MB. If a proxy stands in front of the server, let it pass a request body of at least 27 MB.

# 11 How the system is developed

## From an idea to main

1. **A feature document** in `docs/features/NNN-….md`: requirements, rules, worked examples, acceptance criteria. The number is the highest in `context/spec` + 1.
2. **The pipeline**: `/feature-pipeline docs/features/NNN-….md` in Claude Code. Agents write the specification, the plan, the code and the tests slice by slice; a separate reviewer checks every slice; at the end every criterion is verified, screenshots taken, the full suite run, a PR opened and its checks brought to green.
3. **Your check**: the app from the PR branch is running for you; try the scenarios from the report.
4. **Merging** — only you: `gh pr merge <number> --squash --delete-branch`, then `git switch main && git pull`.

## CI checks on every PR

| Check | What it does |
|---|---|
| Backend | Build, every test with a database container, module boundaries, coverage floor |
| Clients | API contract, formatting, types, tests, lint, web build |
| Security | Leaked secrets (gitleaks), dependency vulnerabilities (pnpm audit, OSV) |
| SAST | Static code analysis (Semgrep) |
| End-to-end | Browser scenarios on a freshly started database |

## Help in the application: source and checks

- The help pages live in `docs/help/uk/<id>.md` and `docs/help/en/<id>.md` (the same `id` and structure). A feature that changes a screen, a button, a permission, a calculation rule or how the platform is run updates the pages in both languages in the same PR. Button names are written only as `{{ui:key}}` from the i18n catalog: a stale key breaks the build.
- The `permissions` field of a page takes `Permission` values and two tokens: `member` (the viewer owns a plot, or holds the permission to view members) and `board` (the viewer holds at least one permission). An empty list means every signed-in user. A page for the board, such as the board's "Getting started", uses `board`; a page for plot owners uses `member`.
- The completeness check: every route and menu item has a page, every screenshot is a file, the two languages match, every `{{ui:…}}` key exists. It runs in the web tests (`make check`) and in the web build (`vite build`): on a mismatch the build fails. In development (`make web`) mismatches are only shown as warnings. To work on part of the pages locally you can run the build with `HELP_CHECK=warn`: then it only warns too. Do not use that flag for a PR.

- Screenshots: `make help-shots ID=<id>` for one page, `make guide-shots` for all of them; the shot list of a page is the `shots` field in its front matter. The script refuses to run against anything but the demo cooperative.
- The board guide PDF is built from the help pages (`make guide`): there is no separate HTML text for it in `docs/guide` any more; the PDF is not edited, the help is.

## Rules

- Agents never merge or approve PRs and never push to `main`; only the owner commits changes to `CLAUDE.md` and `.claude/`.
- Commits never mention AI. All repository text is English, except the guides in `docs/guide` (Ukrainian and English).
- Verification levels: a new feature runs the full suite locally; a small fix on an open PR runs only the affected tests, and CI runs the full suite.
- Models: the orchestrator and the reviewer use your Claude Code session's model (Opus); the worker agents use the model set in `.claude/agents`.
- Working notes outside git: `NOTES.local.md` (questions and answers) and `DECISIONS.local.md` (decisions and deferred tasks).

# 12 Common problems and fixes

| Problem | Cause and fix |
|---|---|
| "Port 8080 was already in use", the web opened on 5174 | A previous run is still going (for example the one the pipeline left for review). `lsof -ti :8080 -ti :5173 \| xargs kill`, then `make up`. |
| "Account not linked" | The email in Keycloak differs from the member card's, or the member is archived, or Keycloak has no `tenant_id`. |
| `admin` cannot sign in to the Keycloak console | Use the **master** realm at `/admin/`, not nadel; check the keyboard layout. |
| "E2E ..." entries and odd plots in the audit log | Traces of old end-to-end tests. Reset the local database (chapter 2); the tests now use a separate database. |
| "platform (linux/amd64) does not match" for postgres | The default PostGIS image is Intel-only and runs emulated on a Mac — slower, tests included. Set `NADEL_POSTGIS_IMAGE` to the arm64 image (chapter 2) and reset the local database. |
| Slow backend tests, long CI | On a Mac first check `NADEL_POSTGIS_IMAGE` (chapter 2): under emulation the suite takes several times longer. The time of every test class is in `backend/target/surefire-reports`. |
| Git: "File exists ... .lock" | A lock file left by an interrupted command. Make sure no git is running and delete the named `.lock`. |
| The backend does not start: `PD_MASTER_KEY_MISMATCH` in the log | The server was started with a master key other than the one that wrapped the cooperatives' keys (a different environment's key, a lost or changed value). Start it with the right key. To change the master key on purpose, start with the new key as `NADEL_PD_MASTER_KEY` and the old one as `NADEL_PD_MASTER_KEY_PREVIOUS` (chapter 8, rotation). Locally: use the key in `infra/.env`, or, if that file was lost, reset the local database with `docker compose -f infra/docker-compose.yml down -v` (demo data only). |
| The backend does not start: `NADEL_PD_MASTER_KEY is missing or is not 32 bytes of base64` | The variable is not set or is malformed (extra characters, a truncated copy). Generate one with `openssl rand -base64 32`, or on a local run use `make up` / `make backend`, which create `infra/.env` (chapter 2). The same message names `NADEL_PD_MASTER_KEY_PREVIOUS` if that one is malformed. |
| Realm file changes did not apply | The realm is imported only into an empty Keycloak — reset the local database. |
| IntelliJ keeps asking for access to other apps' data | System Settings → Privacy & Security → Full Disk Access → IntelliJ IDEA, restart. |

| Problem | Cause and fix |
|---|---|
| A generated Excel file opens with a repair prompt | Tell the developer and attach the template and the generated file. The template filler is our own (ADR 0015); a repair message is one of the conditions for reconsidering it in favour of Apache POI. |
| "The token is not valid" in "Delivery", or the bot stopped answering | The token was mistyped, the bot was deleted or the token reissued in @BotFather. Telegram answers 401 and the system **stops the cooperative's bot until the token is changed**. Copy the token from @BotFather again, save it in "Delivery" and press "Check the bot"; the bot starts working in about a minute. Locally a token starting with "bad" gives this on purpose. |
| The bot is silent although the token is right | One instance polls the bot: if two were running, the second takes it over about a minute after the first stopped - wait. Locally check that `fake-telegram` is running ( `docker compose -f infra/docker-compose.yml ps` ), that `NADEL_TELEGRAM_API_URL` points at it (it does in `dev` by default) and that the demo cooperative's token is not a real one while the address is the fake (and the other way round). When Telegram is unreachable the bot retries with a pause and the application does not fail. |
| "Wrong mailbox login or password" ( `SMTP_AUTH_FAILED` ) | The ordinary password instead of an app password, 2-step verification (Gmail) or access for mail programs (Ukr.net) not turned on, the app password revoked, or the mailbox blocked by the provider for suspicious activity. Create a new app password, save it, send a test letter and press "Send again" on the failed rows. There are no automatic retries, because the password does not fit. For Gmail the screen then shows the app password hint with a link. |
| "This mail server is not allowed" ( `SMTP_HOST_NOT_ALLOWED` ) | The mail server is a local or internal address, or the port is not 25, 465 or 587 (see "The cooperative's mail server: public addresses only"). The chairman enters the server of a mail service, for example `smtp.gmail.com` and 587. There are no retries. Locally this happens only without the `dev` profile. |
| "The mail server refused the letter" ( `SMTP_REJECTED` ) on many letters | A free mailbox reached its daily limit or refuses a send that is too fast. The system retries twice (at once, after 1 and 5 minutes); if it persists, lower "Letters per second" (default 1), wait for the next day and send again. A single refused row is often an invalid recipient address. |
| Telegram answers "too many requests" (429) | Nothing to do: the row is moved to the time Telegram named, the attempt is not used up, and the sending lane pauses for that time. The system keeps its pace below Telegram's limits by itself. |
| No letters visible locally | Open `http://localhost:8025` ; letters are in Mailpit's memory and vanish when the container restarts. Check that the demo cooperative's "Delivery" points at `localhost:1025` . |

| Problem | Cause and fix |
| --- | --- |
| The cash shows a negative balance after a payment was reversed | This is the allowed exception: the money is already given out, so reversing a cash payment is not blocked. The cash card says "The cash balance is negative - do a cash count". Count the cash ("Cash count"); the chairman books the difference as a surplus or a shortage. |
| A bank month is "Needs a new check" again | After "Reconciled", a bank line dated in that month appeared (a payment, an expense, an other income or a difference entered later). The screen says "Reconcile this month again: {kind} of {date} for {amount} was entered on {at}."; the audit log has a "month reopened" entry. Check the statement and press "Reconciled" again. Every later month that was "Reconciled" is reopened the same way (its opening balance changed), with the same line in the notice: check them one by one, starting with the earliest. |
| "Reconciled" stays disabled although the mistake was fixed | The closing balance difference must be zero; differences in the total in or out only warn. If the closing difference is not zero, enter the missing entry or close the remainder with a "Bank difference" for the month (the chairman). A reversal made today is dated today and does not fall into the old month: it gives only a warning. |
| "Invalid period" when building a reconciliation act | The period starts before the plot's first charge (a plot without charges has no period) or ends after today. Enter dates between the first charge and today. |
| A photo is not attached to an expense | The file is over 5 MB, it is the sixth file, or it is not JPEG/PNG/PDF by content. A HEIC photo that the browser cannot open is not attached: the form says "This browser cannot open HEIC photos from an iPhone. Add the photo in Safari, or on the iPhone turn on Settings → Camera → Formats → "Most Compatible" (photos will be JPEG)." A request over 27 MB is rejected by a proxy or the server. |
| The web build fails with an error about the help (a missing page, key or screenshot) | The help completeness check found a mismatch (the error text lists it): add the page or the screenshot, or fix the `{{ui:…}}` key. For partial local work: `HELP_CHECK=warn pnpm --filter web build` (do not use it for a PR). |
| The pipeline stopped | `/feature-pipeline resume` continues where it stopped, from its log. |

# 13 Decision log

| Date | Decision |
|---|---|
| 17.10.2026 | Design: the "calm registry" — side menu, tables, green accent. |
| 18.10.2026 | The chairman's answers: the chairman chooses how common consumers are split; the quarter report shows readings, cost and amount; a building is reference only; no meter, no charge; the chairman configures the meter tree. |
| 18.10.2026 | Other members' plot colours hidden by default; "All plots" only by a general meeting decision. |
| 20.10.2026 | The whole product on the local machine first, servers last. |
| 21.10.2026 | The interface always in one language. A rate has up to 4 decimals, shown without trailing zeros; amounts 2 decimals. |
| 21.10.2026 | A tariff version can be corrected or deleted until a charge uses it; then through recalculation. |
| 21.10.2026 | Readings are not tied to months: a reading is a value on a date; moves in the tree take effect from a date. |
| 22.10.2026 | A resource icon; a plot's two meters in one card; calculated consumption rounded for display. |
| 22.10.2026 | Electricity charged per reading; the chairman chooses the losses mode; negative losses are not charged; common consumers as a separate line. |
| 22.10.2026 | Water like electricity: an unconnected plot pays neither for water nor for the pump; any number of wells and quarters. |
| 22.10.2026 | Verification levels: small fixes run only the affected tests locally, the full suite in CI. |
| 22.10.2026 | Two guides in Ukrainian and English in `main`: for the board (working in the system and the calculation rules) and for the platform owner (this one). |
| 27.10.2026 | Reports introduce no new rules, and templates are data (ADR 0015): our own streaming OOXML filler instead of Apache POI (POI stays the recorded fallback; the engine is about 1900 lines against the threshold of 2000 that triggers a review), with no formula evaluation, no macros and no external links. |
| 28.10.2026 | Payments (015): allocation from the oldest debt, within one debt date by the order of charge types; prepayment goes automatically to new charges (a setting, on by default); cash gets an internal receipt with a running number, not a fiscal receipt; only the chairman reverses a payment; the treasurer splits one payer's payment over several plots when entering it. |

| Date | Decision |
|---|---|
| 28.10.2026 | Documents (016, ADR 0017): PDFs are drawn with Apache PDFBox and an embedded IBM Plex Sans, are always in Ukrainian and are never stored (rendered again from the kept data); the QR code follows NBU format 003 and ZXing; a month's invoices are built while the treasurer waits, without a job queue (JobRunr comes with delivery, feature 017); the ЄДРПОУ code is set by the platform operator; two new permissions, "Issue invoices" and "Issue certificates". |
| 29.10.2026 | Delivery (017, ADR 0018): letters go from the cooperative's own mailbox and Telegram messages through its own bot (no shared platform bot or mail service); their secrets live in the cooperative's data, sealed with its key (the one exception to "secrets only in the environment"); the bot works by long polling with a lease per instance, no webhooks; two database functions that cross cooperatives return ids only; sending is a delivery register worked by JobRunr (three attempts, the pace kept in memory); a new permission "Send announcements". |